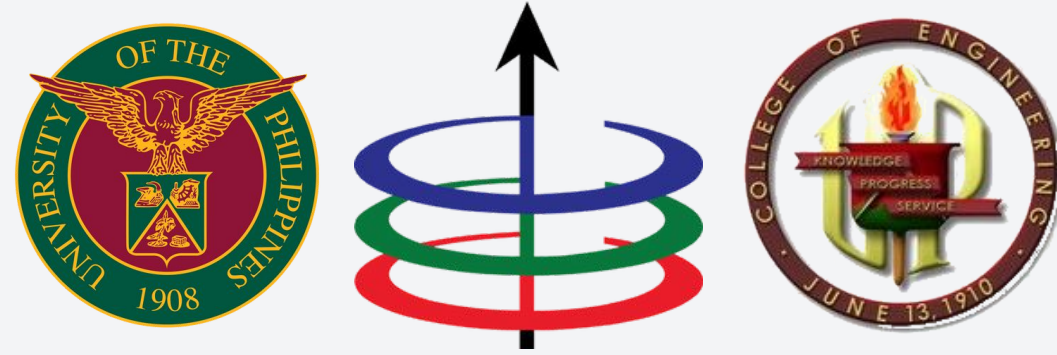




Lecture 1.3

Object Oriented Programming

EEE 121 THVW1

Steven S. Sison

- C++ Classes
- OOP Principles

- **C++ Classes**
- OOP Principles

Suppose we want to create a pokemon storage...

- A pokemon storage system (PC) is a device that stores information about a pokemon including its type, level, etc.
- To do this, we need to find a way to store all of this information in a database or a data structure.



Can we use arrays? Somewhat possible

Method 1: Use multiple arrays.

```
string pokemon[3] = {'pikachu', 'lucario', 'gengar'}
```

```
int level[3] = {55, 76, 89}
```

```
string type[3] = {'electric', 'fighting', 'ghost'}
```

Implementation is easy to understand. However, maintaining it is hard.

- What if I want to add a new pokemon? **Need to resize multiple arrays**
- What if I want to hide my level? **No option to hide it**
- What if I want to print the symbol for electric type? fighting? **Long if-else statements**
- What if I want to add movesets in my pokedex? **More and more arrays!**

C++ Classes and Objects

- Classes are a fundamental part of C++ allowing for the creation of **new user-defined data types**, complete with associated functions and operators.
- In the context of our pokemon example, we can just create a new Pokemon() class as well as other classes for specific pokemons.



```
class Pokemon {  
    protected:  
        // Insert Member Data  
  
    public:  
        // Insert Member  
        // Functions  
};
```

```
class Pikachu : public Pokemon  
{  
    public:  
        // Insert Member  
        // Functions  
};
```

An Object is an instance of a Class

Class

- This is the “blueprint/template” from which you can create an object.
- Combines:
 - Data - identifiers holding a value
 - Methods - code that manipulates data

Object

- An object is an **instance** of a class.
- Uses an identifier called an **object reference**.
 - Essentially the name of the object.
- Example:
 - Ash’s Pikachu is an instance of the Pikachu class.

Class Implementation of Each Pokemon

```
class Pokemon {
```

```
protected:
```

```
    string name;  
    string type;  
    int level;
```

```
public:
```

```
    Pokemon(string n, string t, int lvl) : name(n), type(t), level(lvl) {}  
    void displayInfo();  
    void levelUp();
```

```
};
```

```
void Pokemon::displayInfo(){  
    cout << name << " - Type: " << type;  
}
```

- A class is defined by using the syntax: `class class_name{};`

Class Implementation of Each Pokemon

```
class Pokemon {  
    protected:  
        string name;  
        string type;  
        int level;
```

- A class is defined by using the syntax: `class class_name{};`
- Each class definition should include a **constructor**.
 - A special method that creates an object and assign initial values.

```
    public:
```

```
        Pokemon(string n, string t, int lvl) : name(n), type(t), level(lvl) {}
```

```
        void displayInfo();  
        void levelUp();
```

```
};
```

```
void Pokemon::displayInfo(){  
    cout << name << " - Type: " << type;  
}
```

Class Implementation of Each Pokemon

```
class Pokemon {  
    protected:  
        string name;  
        string type;  
        int level;
```

- A class is defined by using the syntax: `class class_name{};`.
- Each class definition should include a **constructor**.
 - A special method that creates an object and assign initial values.
- Each class has **member data** and **member functions**.

```
    public:
```

```
        Pokemon(string n, string t, int lvl) : name(n), type(t), level(lvl) {}  
        void displayInfo();  
        void levelUp();
```

```
};
```

```
void Pokemon::displayInfo(){  
    cout << name << " - Type: " << type;  
}
```

Class Implementation of Each Pokemon

```
class Pokemon {  
    protected:  
        string name;  
        string type;  
        int level;
```

```
    public:
```

```
        Pokemon(string n, string t, int lvl) : name(n), type(t), level(lvl) {}
```

```
        void displayInfo() const = 0;  
        void levelUp();
```

```
};
```

```
void Pokemon::displayInfo(){  
    cout << name << " - Type: " << type;  
}
```

- A class is defined by using the syntax: `class class_name{};`.
- Each class definition should include a **constructor**.
 - A special method that creates an object and assign initial values.
- Each class has **member data** and **member functions**.
- Each class has **methods**.

Class Implementation of Each Pokemon

```
class Pokemon {  
    protected:  
        string name;  
        string type;  
        int level;  
  
    public:  
        Pokemon(string n, string t, int lvl) : name(n), type(t), level(lvl) {}  
        void displayInfo() const = 0;  
        void levelUp();  
};
```

```
void Pokemon::displayInfo(){  
    cout << name << " - Type: " << type;  
}
```

- A class is defined by using the syntax: `class class_name{};`.
- Each class definition should include a **constructor**.
 - A special method that creates an object and assign initial values.
- Each class has **member data** and **member functions**.
- Each class has **methods**.

Methods and constructors can either be defined inside the class definition or outside the class definition.

- When defined outside, you need to follow the syntax, `class::method`.

Class Implementation of Each Pokemon

An object instance can be created by using the syntax below:

```
class object_name(parameters);
```

**Can you try creating an object instance named
surgingSparksPikachu?**

Class Implementation of Each Pokemon

An object instance can be created by using the syntax below:

```
class object_name(parameters);
```

In the context of our Pokemon class, we can create a new pokemon:

```
Pokemon surgingSparksPikachu("Pikachu", "Electric", 121);
```

The diagram illustrates the mapping of parameters in the instantiation `Pokemon surgingSparksPikachu("Pikachu", "Electric", 121);`. Four red dashed arrows point from labels below to specific parts of the code: from `object name` to `Pokemon`, from `pokemon name` to `surgingSparksPikachu`, from `pokemon type` to `"Electric"`, and from `level` to `121`.

object name pokemon name pokemon type level

Class Implementation of Each Pokemon

An object instance can be created by using the syntax below:

```
class object_name(parameters);
```

In the context of our Pokemon class, we can create a new pokemon:

```
Pokemon surgingSparksPikachu("Pikachu", "Electric", 121);
```

Additionally, we can create a subclass of Pokemons, like a specific class for Pikachu() but we won't be doing that for now. Maybe in the last activity? 😊

Class Implementation of the Pokemon Storage

```
class Pokemon_Storage {  
    private:  
        vector<Pokemon*> = pokemonList;  
  
    public:  
        void addPokemon(Pokemon* p);  
        void showAll();  
};  
  
void Pokemon_Storage::addPokemon(Pokemon* p){  
    pokemonList.push_back(p)  
    cout << p->name << " added\n";  
}  
  
void Pokemon_Storage::showAll() const{  
    cout << "Your Pokemon:\n";  
    for (const auto& p: pokemonList){  
        p->displayInfo()  
    }  
}
```

Here is an implementation of our pokemon storage as a class.

Can you identify which are the constructors? member data? member functions?

Class Implementation of the Pokemon Storage

```
class Pokemon_Storage {  
    private:  
        vector<Pokemon*> = pokemonList;  
  
    public:  
        void addPokemon(Pokemon* p);  
        void showAll();  
};  
  
void Pokemon_Storage::addPokemon(Pokemon* p){  
    pokemonList.push_back(p)  
    cout << p->name << " added\n";  
}  
  
void Pokemon_Storage::showAll() const{  
    cout << "Your Pokemon:\n";  
    for (const auto& p: pokemonList){  
        p->displayInfo()  
    }  
}
```

- In this implementation, the boxed code is the **constructor**.
 - When a Pokemon_Storage() object is initialized, a vector is created to store the pokemons. No need to understand what a vector is, for now.

Class Implementation of the Pokemon Storage

```
class Pokemon_Storage {  
    private:  
        vector<Pokemon*> = pokemonList;  
  
    public:  
        void addPokemon(Pokemon* p);  
        void showAll();  
};
```

```
void Pokemon_Storage::addPokemon(Pokemon* p){  
    pokemonList.push_back(p)  
    cout << p->name << " added\n";  
}  
  
void Pokemon_Storage::showAll() const{  
    cout << "Your Pokemon:\n";  
    for (const auto& p: pokemonList){  
        p->displayInfo()  
    }  
}
```

- Member Data -> Orange Boxes
- Member Functions -> Blue Boxes

Other Side Notes

You might have noticed that we access class attributes differently using: `p->name` and `p.name`.

- `p->name`
 - a. It is essentially `*p.name` (Remember pointers?)
 - b. This is usually used if you are referencing a pointer.
 - c. This is better for memory management, just remember to delete pointers used.

```
void Pokemon_Storage::addPokemon(Pokemon* p){  
    pokemonList.push_back(p)  
    cout << p->name << " added\n";    // p is a pointer based on declaration  
}
```


Other Side Notes

You might have noticed that we access class attributes differently using: `p->name` and `p.name`.

- `p.name`
 - a. Use this when you are accessing the direct object and not a pointer.
 - b. This is simple as no memory management is required, but is limited in terms of flexibility.

```
Pokemon p("Pikachu", "Electric", 5);  
cout << p.name << endl; // Direct access is simple and readable
```

- C++ Classes
- **OOP Principles**

Why follow object-oriented programming?

As the name implies, the main “actors” in the object-oriented design paradigm are the **objects**.

Design goals to be achieved in OOP-based software implementation:

1. Robustness

- a. capable of handling unexpected inputs that are not explicitly defined for its application

2. Adaptability

- a. capable of “evolving” over time in response to changing environmental conditions (e.g. cross-platform usage, easy to patch update)

3. Reusability

- a. can be reused as a component of different systems in various applications

Principles of OOP

1. Abstraction

- a. Show only relevant data and **hide unnecessary details** of an object from the user.
- b. Example: in the string class, we are given the methods (`.len()`, `.append()`), but we do not know their actual implementation i.e. abstracted like a black box.

2. Encapsulation

- a. Binding the data that manipulates it. Keeps data **safe from interference**.
- b. Example: we know that strings are stored in a character array, but when using the string class, we are not allowed to directly modify the character array storing the string.

Principles of OOP

1. Inheritance

- a. The mechanism by which the object acquires the same properties/all properties of an object. These are derived classes, subclasses, or child classes.
- b. Example: In our class implementation of pokemon, the Pokemon() class is the parent class while the Pikachu() class is a subclass. The Pikachu() class **inherits** all properties of the Pokemon() class.

2. Polymorphism

- a. Process objects differently **based on their data type**.
- b. Example: In our class implementation of pokemon, we can print the symbol of the type of the pokemon without the use of if-else statements.

Code Example

In-depth: Abstraction and Encapsulation

```
class Robot {  
    private:  
        string name;  
        double HP;  
        double damage;  
  
    public:  
        Robot(const string &name, double HP, double damage);  
        getName();  
        double getHP();           //  
        string getName();         //  
        double getDamage();       //  
        void introduceSelf();  
        void attackRobot(Robot &rb);  
};
```

- By having a class declaration, we were able to see the member data and member function of the class only.
- Details of constructor and member functions are '**hidden**'

In-depth: Abstraction and Encapsulation

```
class Robot {  
    private:  
        string name;  
        double HP;  
        double damage;
```

```
    public:
```

```
        Robot(const string &name, double HP, double damage);  
        getName();  
        double getHP();           //  
        string getName();         // accessors to get member data  
        double getDamage();       //  
        void introduceSelf();  
        void attackRobot(Robot &rb);
```

```
};
```

C++ access specifiers encapsulates the members:

- **private access**: cannot be accessed or viewed outside the class
- **public access**: can be accessed or viewed anywhere outside the class but within program
- **protected access**: allows derived classes to access members of base class (outside of our scope)

In-depth: Inheritance

Inheritance is an **is-a** relationship between classes.

- Example: Pikachu **is a** pokemon. A honda **is a** car. An orange **is a** fruit.

```
class Pokemon {  
    protected:  
        // Insert Member Data  
    public:  
        // Insert Member  
        // Functions  
};  
  
class Pikachu : public Pokemon {  
    public:  
        // Insert Member  
        // Functions  
};
```

In-depth: Inheritance

Inheritance is an **is-a** relationship between classes.

- Example: Pikachu **is a** pokemon. A honda **is a** car. An orange **is a** fruit.

```
class Pokemon {  
    protected:  
        // Insert Member Data  
    public:  
        // Insert Member  
        // Functions  
};
```

- A derived class can be instantiated as: `class Parent_Class : public Derived_Class{}`

```
class Pikachu : public Pokemon {  
    public:  
        // Insert Member  
        // Functions  
};
```


In-depth: Inheritance

Inheritance is an **is-a** relationship between classes.

- Example: Pikachu **is a** pokemon. A honda **is a** car. An orange **is a** fruit.

```
class Pokemon {  
    protected:  
        // Insert Member Data  
    public:  
        // Insert Member  
        // Functions  
};
```

```
class Pikachu : public Pokemon {  
    public:  
        // Insert Member  
        // Functions  
};
```

- A derived class can be definition as: `class Parent_Class : public Derived_Class{}`
- Member data and functions can be modified to be either public, private, or protected by modifying the definition.

In-depth: Inheritance

Inheritance is an **is-a** relationship between classes.

- Example: Pikachu **is a** pokemon. A honda **is a** car. An orange **is a** fruit.

```
class Pokemon {  
    protected:  
        // Insert Member Data  
    public:  
        // Insert Member  
        // Functions  
};
```

```
class Pikachu : public Pokemon {  
    public:  
        // Insert Member  
        // Functions  
};
```

- A derived class can be definition as: `class Parent_Class : public Derived_Class{}`
- Member data and functions can be modified to be either public, private, or protected by modifying the definition.
- We will not be going more in-depth with inheritance :)

In-depth: Inheritance

Inheritance is an **is-a** relationship between classes.

- Example: Pikachu **is a** pokemon. A honda **is a** car. An orange **is a** fruit.

```
class Pokemon {  
    protected:  
        // Insert Member Data  
    public:  
        // Insert Member  
        virtual void displayInfo()  
};
```

- But! In cases that you want to override some functions from the base class you need to make sure that the function is **virtual** and you use the command **override**.

```
class Pikachu : public Pokemon {  
    public:  
        // Insert Member  
        void displayInfo() override;  
};
```

In-depth: Polymorphism

- Polymorphism refers to the ability of a variable to take different types (i.e. many forms).
- In C++, there are many ways to use polymorphism:
 - Function Template - to use functions for multiple data types
 - Class Template - to create a class that can be used for multiple data types
 - Operator Overloading - to redefine or **morph** how an operator works
- All of these serve the same purpose -> to encourage code reusability.

In-depth: Polymorphism

Application 1: Function Templates

- Suppose we want to create a `min()` function for `int`, `double`, and `float`.

How would you do it?

In-depth: Polymorphism

Application 1: Function Templates

- Suppose we want to create a `min()` function for `int`, `double`, and `float`.
- Intuitively, we create multiple `min()` functions that can be used for each data type.

In-depth: Polymorphism

Application 1: Function Templates

- Suppose we want to create a min() function for int, double, and float.
- Intuitively, we create multiple min() functions that can be used for each data type.

```
int Min_int(int a, int b) {  
    return (a < b ? a : b); // returns the minimum of a and b  
}  
double Min_double(double a, double b) {  
    return (a < b ? a : b); // returns the minimum of a and b  
}  
float Min_float(float a, float b) {  
    return (a < b ? a : b); // returns the minimum of a and b  
}
```

In-depth: Polymorphism

Application 1: Function Templates

- Suppose we want to create a min() function for int, double, and float.
- Intuitively, we create multiple min() functions that can be used for each data type.
- But what if I told you that we can simplify this using function templates?

In-depth: Polymorphism

Application 1: Function Templates

- Suppose we want to create a min() function for int, double, and float.
- Intuitively, we create multiple min() functions that can be used for each data type.
- But what if I told you that we can simplify this using function templates?

```
template <typename T>
T genericMin(T a, T b) {
    // returns the minimum of a and b
    return (a < b ? a : b);
}
```

```
cout << genericMin(3, 4) << ' '
// = genericMin<int>(3,4)
<< genericMin(1.1, 3.1) << ' '
// = genericMin<double>(1.1, 3.1)
<< genericMin('t', 'g') << endl;
// = genericMin<char>('t','g')
```

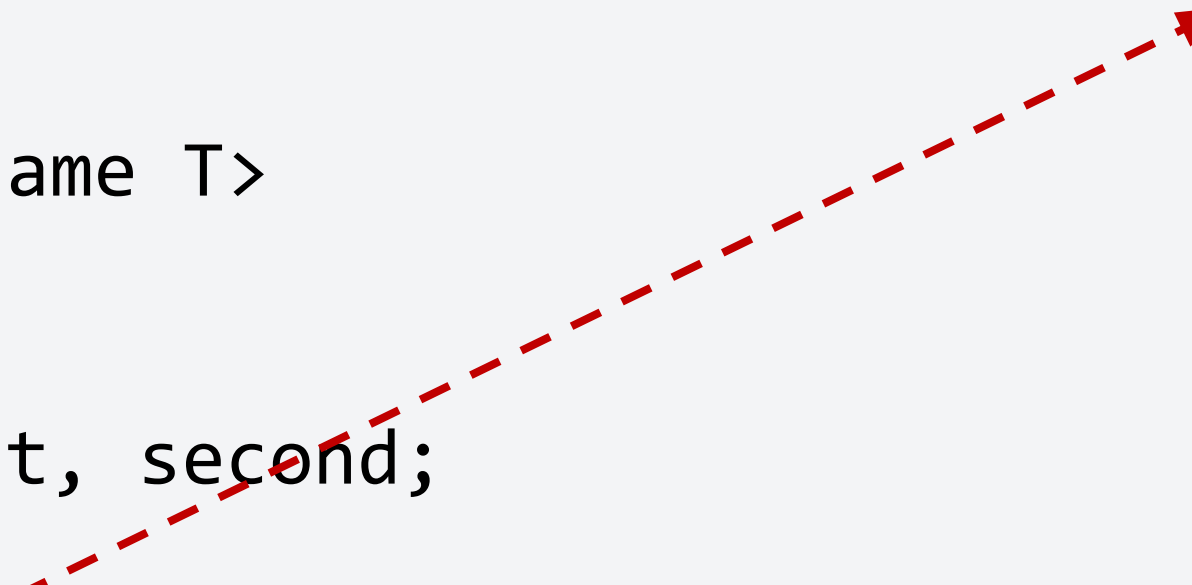
No need for multiple functions!

In-depth: Polymorphism

Application 2: Class Templates

- Consider a class Pair() that uses a class template.

```
template<typename T>
class Pair {
    private:
        T first, second;
    public:
        Pair(T first, T second);
        void show_pair();
};
```



```
template<typename T>
Pair<T>::Pair(T first, T second) {
    this -> first = first;
    this -> second = second;
}
```



```
template<typename T>
void Pair<T>::show_pair() {
    cout << this -> first <<
    " and " << this -> second
    << endl;
}
```

In-depth: Polymorphism

Application 2: Class Templates

- Consider a class Pair() that uses a class template.
- Since the types are not specified, we can use the class Pair() for any data types.

```
template<typename T>
class Pair {
    private:
        T first, second;
    public:
        Pair(T first, T second);
        void show_pair();
};
```

```
Pair<string> p_str =
Pair<string>("Steven", "Sison");
Pair<int> p_int = Pair<int>(3,5);

p_str.show_pair(); // "Steven and Sison"
p_int.show_pair(); // "3 and 5"
```

We have the ability to modify the datatype of the members of the Pair() class.

In-depth: Polymorphism

Application 3: Operator Overloading

- We can also **morph** or redefine how an operator works when applied to a class.

In-depth: Polymorphism

Application 3: Operator Overloading

- To do this, use this syntax. a is of class A, b is of class B, @ is the operator.

Expression	Operator	Member Function
@a	+-*&!~ ++ --	A::operator@()
a@	++ --	A::operator@(int)
a@b	+ - * / % ^ & < > == != <= >= << >> && ,	A::operator@(B)
a@b	= += -= *= /= %= = ^= &= <<= >>= []	A::operator@(B)
a(b,c, ...)	()	A::operator()(B,C, ...)
a -> b	->	A::operator->()
(TYPE)a	TYPE	A::operator TYPE()

In-depth: Polymorphism

Application 3: Operator Overloading

- Using the Pair() class from before, we want to modify the “+” operator to perform element-wise addition for the pairs.

Any ideas?

In-depth: Polymorphism

Application 3: Operator Overloading

- Using the Pair() class from before, we want to modify the “+” operator to perform element-wise addition for the pairs.

```
template<typename T>
class Pair {
    private:
        T first, second;
    public:
        Pair(T first, T second);
        Pair<T> operator+(Pair B);
        void show_pair();
};
```

```
template<typename T>
Pair<T> Pair<T>::operator+(Pair b) {
    return Pair(this -> first + b.first,
                this -> second + b.second);
}
```

We have defined how ‘+’ operator works on ‘Pair’ class.

In-depth: Polymorphism

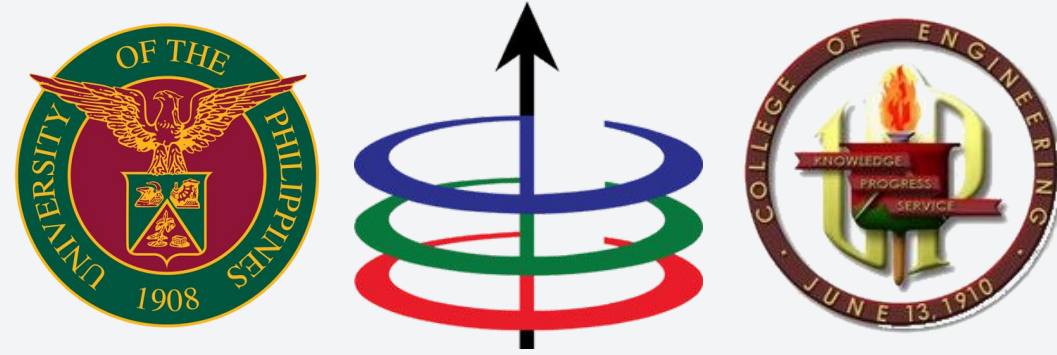
Application 3: Operator Overloading

- Using the Pair() class from before, we want to modify the “+” operator to perform element-wise addition for the pairs.
- If we try using it in main() now, we see this.

```
Pair<string> p_str1 = Pair<string>("Steven", "Sison");  
Pair<string> p_str2 = Pair<string>("3", "5");  
Pair<string> p_str3 = p_str1+p_str2;  
p_str3.show_pair(); // “Steven3 and Sison5”
```

Since both p_str1 and p_str2 are strings, the first element of each pair gets concatenated together. The same happens with the second element.

Any questions?



Lecture 1.3

Object Oriented Programming

EEE 121 THVW1

Steven S. Sison